Do We Still Need to Be Agile? What Student Projects Reveal About Coordination Skills in the GenAI Era

Anonymous Authors

***Abstract*—Generative AI can make local software implementation faster, but it may also raise the importance of shared architectural intent, integration, and human coordination. This study examines that tension across two cohorts of student software projects comprising 14 teams and three synchronized checkpoints (T_1/T_2/T_3). We combine multi-checkpoint surveys, repository mining, independent blind-scored evaluator scorecards, and transcript-based evidence to study Generative AI adoption and role perceptions, the temporal density of repository activity, and associations between upfront architectural artifacts, late rework, and project outcomes. The results show that self-reported AI use is widespread and task-oriented, but perception families do not form a single longitudinal trajectory. Repository activity and clean source/test change concentrate near final integration, but internal contrasts show that this late concentration includes cases consistent with compensatory recovery, cases without evaluator-score gain, and cases where improvement occurs with more distributed activity. Candidate coordination-language density increases across the observed transcript checkpoints, while observable T1 architectural-artifact scope varies substantially without a simple monotonic association with T3 clean-rework magnitude, rework ratios among baseline-eligible cases, or independent outcomes. Robustness checks using non-overlapping phases, operational-regularity proxies, influence diagnostics, complexity profiles, and explicit transcript coverage make the main validity threats inspectable but do not remove the study's observational limits. These findings suggest that Agile remains necessary, but AI-assisted development requires a more explicit and inspectable layer of architectural planning, interfaces, dependencies, and acceptance conditions. Spec-Driven Development is discussed as a plausible, testable response to the coordination gap exposed by the combined timing, planning, rework, complexity, and influence diagnostics.**

I. Introduction

The Agile Manifesto revolutionized software engineering by prioritizing working software, individual interactions, and responding to change over comprehensive documentation and strict upfront planning. For over two decades, this empirical paradigm was the gold standard for navigating requirements uncertainty, and for good reason: in a world where local implementation effort and changing requirements made heavyweight plans costly, minimizing unnecessary upfront documentation helped reduce waste.

That world is changing. Generative AI assistants now produce substantial amounts of code in seconds, shifting scarce software-engineering effort toward architectural alignment, integration, and cognitive coordination. Under this new cost equation, the Agile bet—that planning can remain deliberately light as understanding develops—deserves to be tested rather than assumed.

If deferring design in an AI-assisted environment merely relocates effort, late project activity may reflect several mechanisms: compensatory recovery, delayed integration without assessed improvement, distributed progress, or technical-complexity pressure. This paper examines how such mechanisms become visible through repository metadata, multi-checkpoint student surveys, independent evaluator scorecards, and qualitative meeting transcripts across two cohorts of software-engineering projects. The resulting evidence connects what students report, what repositories reveal, what evaluators score, and what coordination language appears in the project record.

We structure the investigation around three questions:

- **RQ1 (Descriptive):** How do student developers utilize Generative AI, and how does this adoption shape their longitudinal perceptions of software engineering roles and project expectations?
- **RQ2 (Descriptive):** How does the temporal density of repository activity contrast with the qualitative typification of human coordination friction across the project lifecycle?
- **RQ3 (Primary):** How does the observable scope and content evidence of upfront architectural artifacts at baseline T_1 associate with late-stage clean-path rework and independent, blind-scored project outcomes?

The study makes three contributions. First, it provides a longitudinal, multi-method account of AI-assisted student development that keeps perceived AI impact, self-reported adoption, repository activity, and qualitative coordination evidence analytically distinct. Second, it makes the temporal contrast between repository activity and integration pressure observable and qualifies that contrast using non-overlapping phase bins, timing/outcome contrasts, operational regularity, and coverage-aware transcript evidence while preserving the different grains and coverage of the underlying evidence. Third, it examines upfront architectural-artifact evidence, late clean-path rework, and independent outcomes without collapsing structural observations into an unsupported quality score. The resulting picture is deliberately more useful than a simple success/failure verdict: rather than eliminating threats such as ordinary deadline pressure, process heterogeneity, small-sample sensitivity, and technical complexity, it makes them

explicit and inspectable. It shows where implementation speed meets the harder problem of shared system intent.

The findings motivate a practical proposition. Generative AI does not remove the need for Agile practice; it increases the importance of making architectural intent, interfaces, dependencies, and acceptance conditions explicit while remaining open to revision. These observations motivate Spec-Driven Development (SDD), understood here as using an explicit specification as the shared reference for planning, tasks, implementation, and checks, as a theoretically grounded, testable hypothesis: carrying that specification through technical planning, task decomposition, implementation, and convergence checks may help teams address the coordination bottlenecks exposed by the combined timing, planning, rework, complexity, and influence diagnostics [1], [2]. This observational study neither implemented SDD nor compared it with another development approach; therefore, it does not establish SDD's effectiveness. Instead, the hypothesis invites future intervention research into how teams can preserve adaptability while ensuring that people and AI systems act on the same inspectable account of what the system is meant to become.

II. Background and Related Work

Agile development prioritizes working software, interaction, and response to change over comprehensive documentation [3], but that value hierarchy does not remove the need for shared architectural intent. Classic coordination accounts emphasize that communication costs grow with system size and team interaction when structure remains tacit [4], [5]. The GenAI-era question is therefore not whether teams should return to immutable upfront design, but which intent, interface, dependency, and acceptance constraints must remain explicit while implementation iterates.

Generative AI changes local software-engineering work unevenly across tasks, users, and contexts [6]–[10]. Early evaluations of code-generating models examine their capabilities [11], while studies of developer interaction show that programmers steer, verify, and contextualize generated code rather than merely accepting it [12]. Critical perspectives also warn that locally plausible outputs can obscure system-level context [13]. For student projects, this distinction matters because faster local generation does not automatically resolve cross-file dependencies, architectural boundaries, or coordination across contributors. Studies of student experiences document GenAI use in undergraduate programming and software-engineering education [14], [15], while project-course research describes the coordination demands of client-facing software engineering work [16].

Architecture and technical-debt research similarly treats design decisions as evolving constraints whose consequences may accumulate over time [17]–[22]. This motivates measuring early architectural artifacts separately from semantic quality: an observed artifact is evidence of an observable practice, not proof that the design was adequate. Spec-Driven Development (SDD) extends this concern into AI-assisted workflows by carrying a specification through planning, task decomposition, implementation, and convergence checks [1], [2]. We treat SDD as a testable process proposition, not as an already validated intervention.

The gap addressed here is how these perspectives meet in student projects that use GenAI while being observed across synchronized checkpoints. The study combines survey reports, repository activity and path provenance, independent evaluator scorecards, and transcript-based coordination evidence. Its contribution is not a causal estimate of AI productivity or a validation of SDD; it is an evidence-preserving account of how local AI-assisted implementation, temporal integration pressure, explicit artifacts, rework proxies, and independent outcomes can be examined together without merging their analytical grains.

III. Methodology

We studied two offerings of a project-based undergraduate software-engineering course at a large public university in South America: 2025.2 ($N = 9$ analyzable teams) and 2026.1 ($N = 5$), for $N = 14$ team-semesters and approximately 70 participating students. Teams of roughly five students had autonomy over process model and project management approach, including Scrum, Kanban, other Agile variants, or unguided work. Each team built an operational software project with an integrated GenAI component, but no AI architecture was mandated. Retrieval-Augmented Generation (RAG), in which a model conditions its output on retrieved project or domain documents, was presented as one domain-dependent example [23]; none of the 14 projects implemented a full RAG pipeline. The analyzed systems spanned healthcare, finance, education, research, ecotourism, onboarding, and tutoring domains, and used project-specific integrations such as summarization, document/profile analysis, chatbots, recommendations, and content generation.

We report this observational, multi-method study with reference to established empirical software-engineering research and case-study guidance [24], [25].

The course used three synchronized checkpoints. T_1 was administered on the first day of class, before formal instruction on GenAI processes; T_2 followed modules on GenAI mechanics, RAG architectures, and software process; and T_3 occurred one week before final delivery and presentation. The study combines four evidence streams. First, complete Git histories provide commits, line changes, file provenance, branch information, and repository-visible artifacts. Students were instructed to keep requirements, architecture, documentation, and source code in the repository, but Git captures only repository-visible evidence–artifacts and changes committed to version control–rather than AI-use telemetry such as prompt logs, API calls, or IDE acceptance events. Second, independent evaluator scorecards were collected at T_1–T_3 through blind scoring after status presentations, with 4 evaluators in 2025.2 and 3 in 2026.1. Third, multi-checkpoint surveys captured demographics, experience, GenAI adoption, perceived role disruption, and project expectations at cohort/checkpoint grain; survey responses carry no team identifier. Fourth, a bounded

2025.2 transcript corpus covers status presentations and supervisor discussions, segmented into 100 chunks across T_1, T_2, and T_3. Transcript evidence is a global checkpoint corpus and is not imputed for 2026.1.

All raw streams were anonymized before analysis. Analytical artifacts exclude direct personal identifiers; restricted provenance records preserve prompts, responses, hashes, and usage metadata for auditability.

A. Analytical Approach

The analysis is descriptive and preserves the scale and grain of each evidence stream. Means are reported with standard deviations for approximately interval-like scores; medians and interquartile ranges are used for skewed counts and ratios. Proportions retain denominators. Rolling seven-day windows are indexed relative to evaluator anchors and are temporal summaries, not independent observations. Exploratory associations use Spearman's rank correlation ρ [26], which tests whether two variables tend to move monotonically together after replacing raw values by ranks. This choice avoids assuming linear relationships or normally distributed variables in a small, mixed-scale panel. Leave-one-out analyses recompute associations after removing each eligible team-semester.

In addition to the primary M1–M9 families, robustness views include non-overlapping phase bins, final-seven-day concentration contrasts by score change and planning scope, operational-regularity proxies, technical-complexity profiles, and leave-one-out sensitivity ranges. *Epistemological boundaries and grain separation:* Because this study observes $N = 14$ team-semesters and approximately 70 students without a non-AI control group or IDE-level telemetry, three principles govern interpretation of the Section IV results. First, *strict grain preservation*: student- level survey waves (M1/M2), global checkpoint transcripts (M5), and team-semester repository/scorecard metrics (M3, M4, M6–M9) retain distinct denominators (Table I) and are never pooled or cross-correlated across incompatible grains. Second, *exploratory inference*: Spearman correlations (ρ), p-values, phase bins, final-seven-day contrasts, operational-regularity proxies, technical-complexity stratifications, and leave-one-out ranges (which show sensitivity to influential cases rather than confidence intervals) are descriptive or robustness diagnostics, not confirmatory tests or causal estimators. Third, *proxy transparency*: missing repository or transcript observations remain explicit observation states rather than being zero-filled or imputed as low quality. These boundaries are stated here and in Section VI rather than repeated for every metric.

B. Metric Families

RQ1 is operationalized at cohort/checkpoint grain. Self-Reported AI Usage and Perceptions (M1) separates six perception families from self-reported usage frequency, tasks, tools, prior experience, autonomy/tool balance, and exploratory career topics. Perceived Role Disruption (M2) preserves role-specific Likert distributions and paired sensitivity summaries. Neither metric is behavioral AI-use telemetry.

RQ2 uses repository and transcript evidence with explicit grain separation. Repository Activity (M3) summarizes commit timing, author participation, rolling windows, final-seven-day concentration, and non-overlapping phase shares for 14 team-semesters. Clean-Change Dynamics (M4) applies a versioned clean-path policy and reports clean churn separately from total churn. Coordination-Language Evidence (M5) reconstructs one 2025.2 global transcript corpus per checkpoint and deterministically counts versioned lexical markers; it is not correlated with team-semester repository observations.

RQ3 uses team-semester artifacts and outcomes. Upfront Architecture Evidence (M6) combines repository-visible T1 artifact presence/scope (M6a) with structured Planning Content Evidence (M6b) from T1 commit subjects for nine observed team-semesters; five cases without T1 subjects remain unavailable. Repository Inactivity (M7) reports timing and coverage diagnostics, not planning omission. Clean-Path Rework (M8) reports late changes to previously observed clean source/test paths, using file provenance as a rework proxy while preserving baseline eligibility. Artifact–Rework–Outcome Associations (M9) reports exploratory Spearman associations among T1 artifact predictors, M8 rework, and independent T3 scorecard outcomes, with ratio analyses restricted to baseline-eligible cases. Technical-complexity profiles contextualize rework and concentration, but are not causal adjustments.

C. Language-Model Assistance

Language-model assistance is used for bounded data extraction and is kept separate from deterministic metric construction. M1 uses coded survey responses and M5 uses deterministic lexical coding; neither receives a new language-model score. M6b uses structured extraction from T_1 commit subjects to identify goals, architecture/design, task decomposition, risk/dependency, and literal evidence quotes. Outputs were reviewed by the research team and retained as separate exploratory categories, never as a composite planning-quality or architectural-quality score.

IV. Results

This section reports descriptive and exploratory results from the validated metric artifacts, organized by research question.

A. RQ1: How do student developers utilize Generative AI, and how does this adoption shape their longitudinal perceptions of software engineering roles and project expectations?

RQ1 is supported by two distinct evidence streams: Self-Reported AI Usage and Perceptions (M1), which summarizes six perception families and separately reports self-reported AI use, and Perceived Role Disruption (M2), which summarizes perceived disruption agreement for six software-engineering roles. All six cohort-by-semester/checkpoint cells in the M1 family have complete item coverage, with 46, 41, and 40 respondents in 2025.2 and 22, 20, and 18 respondents in 2026.1 at T_1, T_2, and T_3, respectively.

TABLE I
SUMMARY OF METRIC FAMILIES, OPERATIONAL GRAINS, AND COVERAGE

ID	Metric Family and Components	Primary Grain (N)
M1	Self-Reported AI Usage & Perceptions (6 perception scales + task/tool frequency)	Cohort $\times$ checkpoint (6 cells)
M2	Perceived Role Disruption (5-point Likert across 6 SE roles + paired shifts)	Cohort $\times$ checkpoint $\times$ role
M3	Repository Activity (commit timing, rolling windows, author Gini, phase bins)	Team-semester / window ($N = 14$)
M4	Clean-Change Dynamics (source/test line churn under clean-path policy)	Team-semester / checkpoint ($N = 14$)
M5	Coordination-Language Evidence (deterministic lexical markers per 1k tokens)	Global corpus / checkpoint (`2025.2`)
M6	Upfront Architecture Evidence: artifact scope (M6a, $N = 14$) & commit-subject categories (M6b, $n = 9$)	Team-semester ($N = 14$ / $n = 9$)
M7	Repository Inactivity (7-day window coverage diagnostic)	Team-semester / window ($N = 14$)
M8	Clean-Path Rework (late modifications to prior clean source/test files)	Team-semester ($N = 14$; $n = 12$ ratio)
M9	Artifact–Rework–Outcome Associations (Spearman ρ + leave-one-out sensitivity)	Team-semester / stratum ($N = 14$)

The six perception families do not move as a single trajectory. Mean AI benefit increases in 2025.2 but declines in 2026.1, while autonomy/tool balance increases in both cohorts and project-expectation families remain comparatively low. These item families retain their original scales and should not be interpreted as one latent measure of AI dependence. Figure 1 presents these six families with cohort-specific standard-deviation bars; exact endpoint means and standard deviations are available in the published data explorer [27].

The raw usage fields confirm broad, task-oriented adoption. Daily general AI use and project-specific use remain common across cohorts, with reported tasks concentrated in research/questions, code generation, debugging, and documentation. Tool naming is dominated by widely available assistants such as ChatGPT and Gemini, while prior AI-project experience increases across checkpoints.

Career-impact topic coding is retained as an exploratory supplement rather than a validated result. Its largest categories are uncertainty/conditional statements and other/insufficient evidence, so the rule-based proportions are presented as descriptive context only.

Perceived Role Disruption (M2) shows a role-specific pattern rather than a uniform increase in perceived disruption. Frontend tends to receive higher agreement scores, whereas project-management roles are not consistently high across checkpoints. Privacy-preserving paired summaries also move in mixed directions rather than showing a uniform displacement trajectory. Agreement is therefore role-specific and cohort-dependent; it does not support a general claim that all roles become progressively more exposed to replacement or transformation.

B. RQ2: How does the temporal density of repository activity contrast with the qualitative typification of human coordination friction across the project lifecycle?

RQ2 contrasts repository activity (M3), clean source/test change (M4), and global transcript coordination markers (M5) across checkpoints (Table II).

Repository Activity (M3) shows a pronounced concentration around the final evaluator anchor. The semester-pooled rolling view rises toward the final checkpoint, peaks immediately after it, and then falls sharply, producing a clear late-stage bottleneck. These rolling windows overlap, so adjacent points are temporal summaries rather than independent observations. Figure 2 visualizes this pooled temporal shape, and Table II reports the checkpoint values and units.

The checkpoint participation view shows the same concentration with substantial heterogeneity. Across the 14 team-semesters, 7 have any post-checkpoint activity. The final-window share of all commits ranges from 9.5% to 100.0%; three team-semesters have fewer than ten total commits, and their shares should be treated as small-denominator descriptions. In the author concentration output, the median pre-anchor window contains 12.5 commits, 3.5 active days, and 3 authors, with a median maximum-author share of 0.50 and median Gini coefficient of 0.22. Here, the Gini coefficient summarizes how unevenly commits are distributed across authors, with values near zero indicating more even participation and larger values indicating stronger author-level concentration. Post-anchor windows are frequently inactive; where activity exists, concentration statistics are not interpreted for no-activity cases.

The same final-stage pattern remains visible when the 91 days before T_3 are re-expressed as mutually exclusive phases rather than overlapping rolling windows. These phase-share summaries are available in the public data explorer [27] as descriptive checks against the overlap structure of the rolling M3 view.

To address whether the final peak is simply "student syndrome," we compare final-seven-day concentration–the share of a team's total repository activity occurring in the seven days around final delivery–with evaluator-score change and repository-visible planning scope at team-semester grain ($n = 14$). Higher final concentration appears in both positive-delta and non-positive-delta cases, and the clean-churn variant agrees directionally with the commit-based view. Figure 3 keeps the representative planning/concentration contrast in the

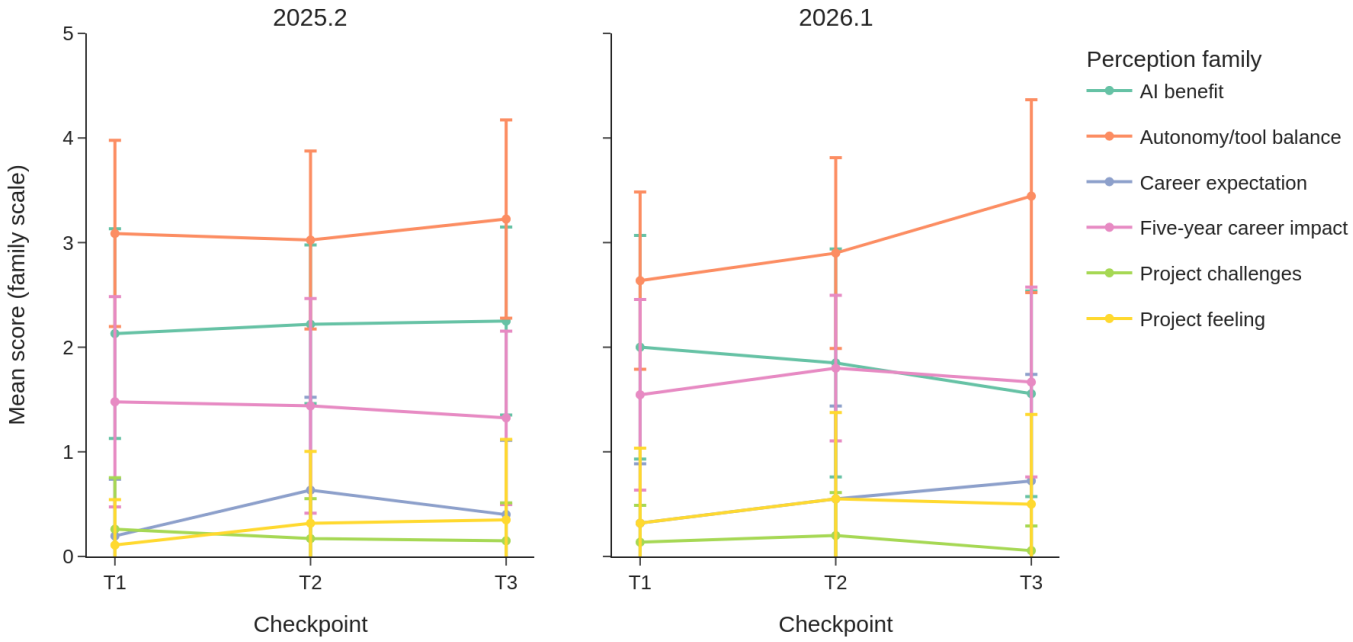


Fig. 1. Self-Reported AI Usage and Perceptions (M1) by cohort and checkpoint. Points show means and bars show one standard deviation; family scales are retained from the underlying instruments.

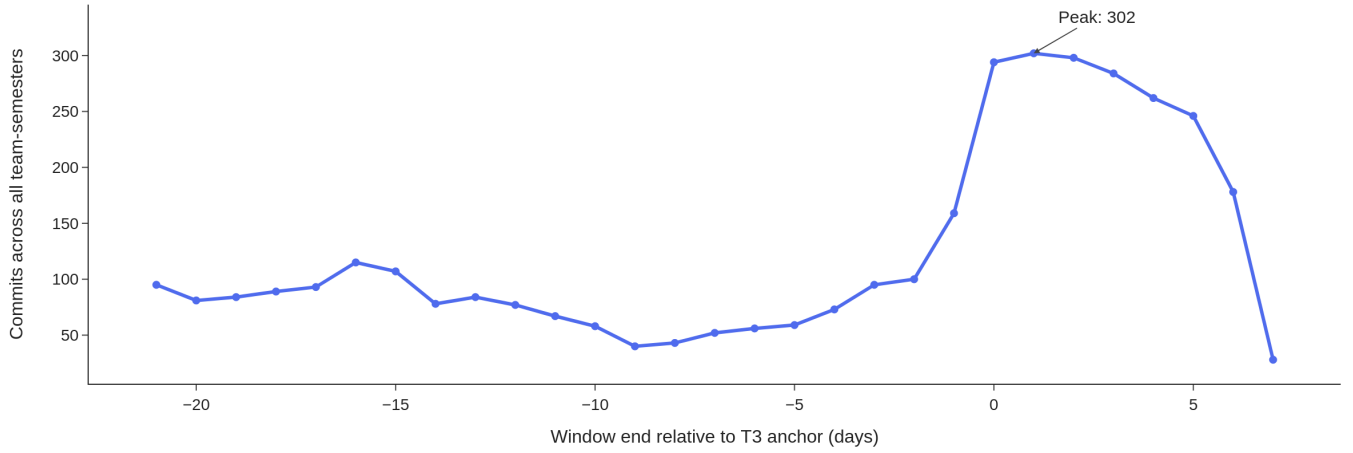


Fig. 2. Repository Activity (M3) in pooled rolling windows. The vertical marker identifies the T_3 evaluator anchor; the annotation identifies the pooled peak.

main text; the score-delta and clean-churn variants are also available in the public data explorer [27].

Operational regularity provides a final repository-visible process check. The regularity index is descriptive, with larger values indicating activity more distributed across the observed Git timeline; it is not a validated agile process scale. Both low and high final concentration appear across the observed regularity range.

Clean-Change Dynamics (M4) reproduces the temporal increase using a narrower clean-change definition. Pooled clean churn rises from 21,718 lines across 7 observed team-semesters at T_1, to 47,545 lines across 10 at T_2, and 136,325 lines across all 14 at T_3. The median clean churn per team-semester rises from 66.5 lines at T_1 to 3,232.5 at T_2 and 6,975.5 at T_3. Clean change accounts for 42.9%, 19.6%, and 1.0% of all recorded churn at the three checkpoints, respectively, because excluded or non-measurement paths dominate the raw line-change total in the final checkpoint. This distinction is why Clean-Change Dynamics (M4) is not interchangeable with total repository activity.

Coordination-Language Evidence (M5) provides a separate textual axis for the 2025.2 transcript corpus. The corpus contains 35, 37, and 28 transcript chunks at T_1, T_2, and T_3, respectively, with two source sessions at each checkpoint.

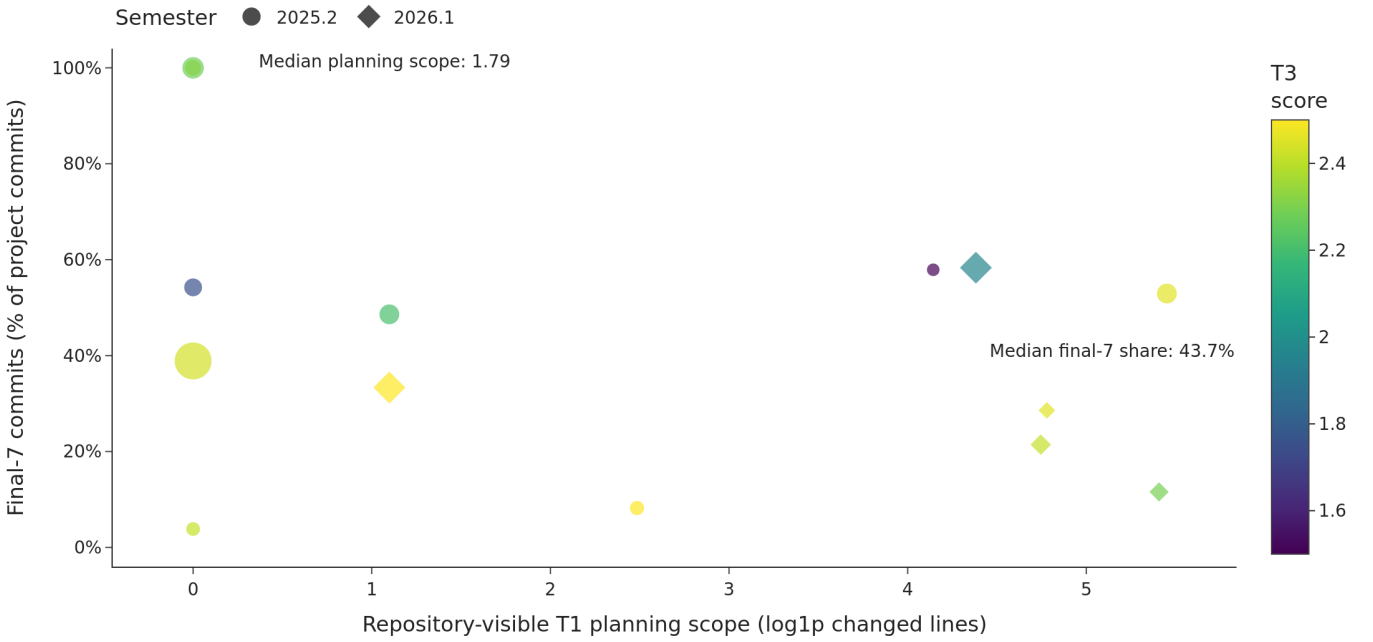


Fig. 3. Repository-visible planning scope versus final-seven-day commit concentration. Marker color reports T3 evaluator score and marker size reports absolute score change; quadrants are descriptive median splits.

Candidate marker density increases from 0.515 to 1.132 and 1.287 markers per 1,000 transcript tokens. Marker counts are 12, 54, and 40, over 23,296, 47,705, and 31,087 tokens. The composition also changes: integration markers account for 20 of 54 markers at T_2, while blockers account for 7 of 40 at T_3. These are deterministic lexical matches in a global corpus, not diagnoses of coordination failure; transcript coverage is unavailable for 2026.1.

Table II consolidates the principal RQ2 values while retaining their distinct units and analytical grains. Figure 4 shows the Clean-Change Dynamics (M4) and Coordination-Language Evidence (M5) checkpoint trajectories in separate panels.

Repository activity and clean source/test change (M3/M4) concentrate near the final checkpoint, while candidate coordination-language density (M5) increases across the observed 2025.2 transcript corpus.

C. RQ3: How does the observable scope and content evidence of upfront architectural artifacts at baseline T_1 associate with late-stage clean-path rework and independent, blind-scored project outcomes?

RQ3 evaluates team-semester associations ($N = 14$) among upfront architectural artifacts at T_1 (M6a, M6b), late clean-path rework (M8), and independent blind-scored outcomes at T_3 (M9).

Architectural Artifact Scope (M6a) records whether a T1 architectural artifact was observed and its repository-visible structural scope. An artifact is present in 4 of 9 team-semesters in 2025.2 and in all 5 team-semesters in 2026.1. Among the 2025.2 observations, the median T1 file count is zero and the median T1 line delta is zero; among the 2026.1 observations, the corresponding medians are 4 files and 114 changed lines. The largest observed T1 line delta is 232 in 2025.2 and 222 in 2026.1. These are structural observations, not measures of architectural quality or planning intent. Figure 5 summarizes the cohort-level presence pattern.

Planning Content Evidence (M6b) adds structured evidence from T1 commit subjects for 9 of the 14 team-semesters; 5 cases have no T1 subjects and were not measured. The reviewed fields cover planning evidence, goals, architecture/design, task decomposition, risk/dependency, and literal evidence quotes. They are retained as separate exploratory fields, with no composite planning or architectural-quality score.

Repository Inactivity (M7) shows that repository inactivity is not equivalent to architectural-artifact absence. At T_1, 7 of 9 team-semesters are inactive in the 2025.2 seven-day window and 3 of 5 are inactive in 2026.1; by T_3, no team-semester is inactive in either cohort. Repository Inactivity (M7) is therefore used as a repository timing and coverage diagnostic, not as a planning-omission measure and not as a predictor in Artifact–Rework–Outcome Associations (M9).

Clean-Path Rework (M8) reports late-stage changes to previously observed clean paths as a file-provenance proxy for destructive rework. Among the 14 team-semesters, 12 are baseline-eligible. The T3 clean-rework ratio ranges from 0.000 to 1.000 among the reported observations, while clean rework magnitude ranges from zero to 15,518 lines. For example, the largest magnitude occurs in 2025.2/TEAM_09, whereas 2025.2/TEAM_08 and 2026.1/TEAM_03 both exhibit

TABLE II
RQ2 TEMPORAL SUMMARY WITH EXPLICIT UNITS

Metric	Unit / grain	T_1 or -7	T_2 or -1	T_3 or 0	Post-anchor	Coverage
Repository Activity (M3)	Pooled commits / rolling window	52 (-7)	159 (-1)	294 (0)	302 ($+1$); 28 ($+7$)	14 team-semesters
Clean-Change Dynamics (M4)	Clean churn lines / checkpoint	21,718	47,545	136,325	–	7 / 10 / 14 observed
Coordination-Language Evidence (M5)	Markers per 1,000 tokens / global corpus	0.515	1.132	1.287	–	2025.2 only

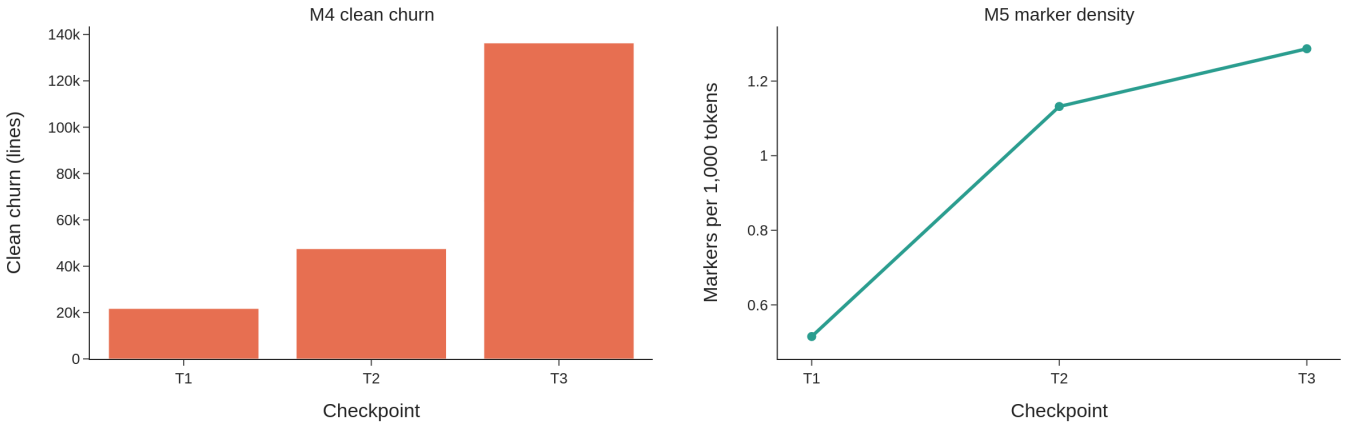


Fig. 4. RQ2 checkpoint summaries with explicit units. Clean-Change Dynamics (M4) and Coordination-Language Evidence (M5) use separate panels and scales.

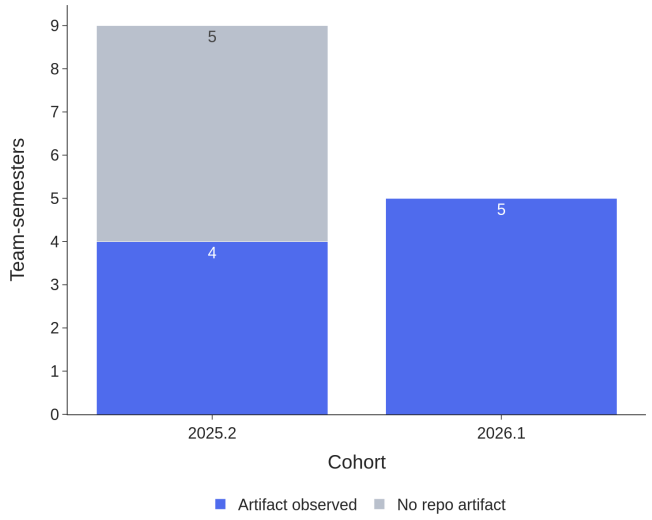


Fig. 5. T1 architectural-artifact presence by cohort. "No repository artifact observed" is an observation state, not evidence that planning did not occur elsewhere.

a rework ratio of 1.000, but their absolute magnitudes differ substantially (172 and 196 lines). Baseline-ineligible cases remain separate and are not interpreted as stable or low-rework cases. Because path provenance does not establish semantic defects, Clean-Path Rework (M8) is not a direct defect measure. Figure 6 displays the artifact-scope/rework profile on a logarithmic rework axis, preserving baseline eligibility.

Technical complexity is a concurrent descriptive explanation for rework and late concentration. The median T3 technical-complexity evaluator score is 1.875 in the 14 team-semesters. When associations are stratified by this median, planning scope retains a positive association with clean-rework magnitude in both strata ($\rho = 0.45$ in the lower-complexity stratum and $\rho = 0.26$ in the higher-complexity stratum), but several other directions change after stratification. For example, final-seven-day clean-churn concentration versus clean-rework magnitude changes from $\rho = -0.03$ in the unstratified summary to $\rho = -0.47$ in lower-complexity cases and $\rho = 0.54$ in higher-complexity cases. These splits are descriptive small-n sensitivity checks, not regression adjustment. Figure 7 visualizes the complexity profile used for this alternative explanation.

Table III summarizes M9 associations across the 14 team-semesters. T1 artifact scope has weak, non-monotonic associations with T3 rework; ratio estimates are limited to baseline-eligible cases, and independent T3 outcomes show similarly weak associations. In the separate M6b subset ($n = 9$), task-decomposition evidence has the largest absolute association with clean-rework magnitude ($\rho = -0.408$) and the largest positive association with an evaluator outcome, scope applicability ($\rho = 0.479$). Leave-one-out and broader influence-map views expose case sensitivity; these coefficients and p-values are exploratory diagnostics, not confirmatory tests or causal estimates, and leave-one-out ranges are not confidence intervals. Further sensitivity results are available in the public data explorer [27].

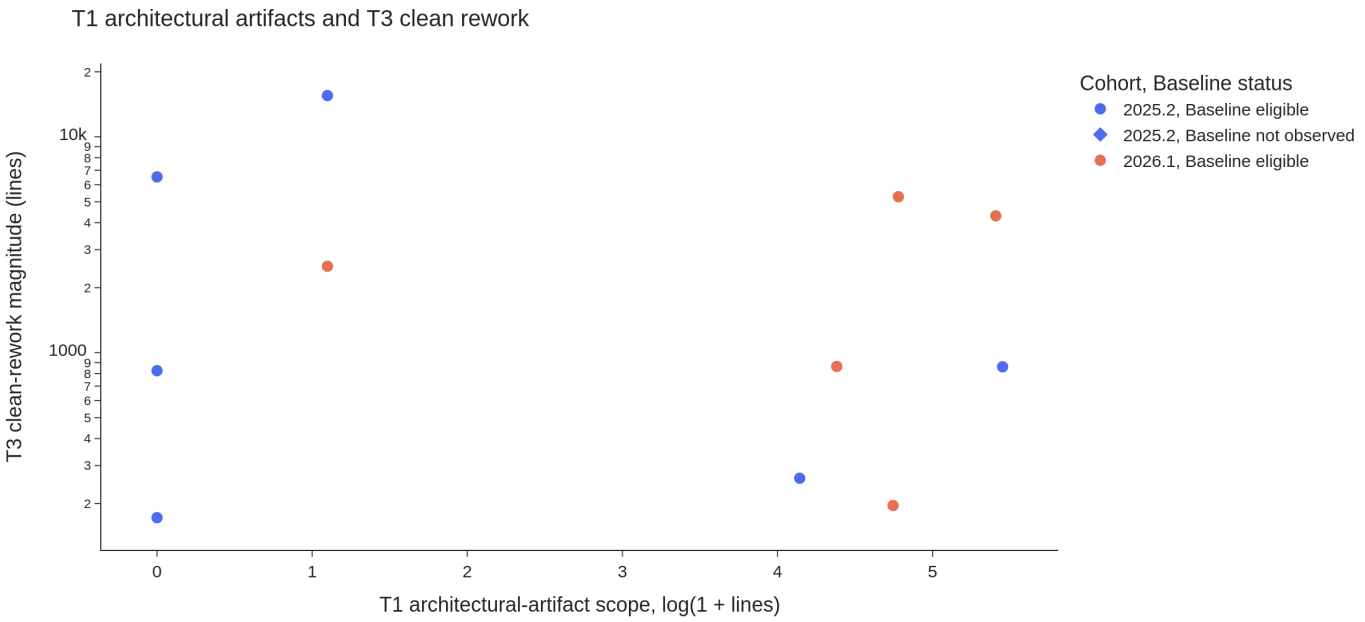


Fig. 6. T1 architectural-artifact scope versus T3 clean-rework magnitude. Points represent team-semesters; the vertical scale is logarithmic so that small and very large rework magnitudes remain visible in the same panel. Clean rework is a file-provenance proxy, not semantic defect validation.

TABLE III
PRIMARY RQ3 ASSOCIATION RESULTS

Predictor (T_1)	**Outcome** (T_3)	**Stratum**	n	**Spearman** ρ	**Expl.** p
T_1 artifact file count	T_3 rework magnitude	All team-semesters	14	0.206	0.479
T_1 artifact line scope	T_3 rework magnitude	All team-semesters	14	0.301	0.296
T_1 artifact file count	T_3 rework ratio	Baseline-eligible	12	−0.032	0.922
T_1 artifact line scope	T_3 rework ratio	Baseline-eligible	12	−0.060	0.853
T_1 artifact file count	Project progress	All team-semesters	14	0.100	0.733
T_1 artifact file count	Scope applicability	All team-semesters	14	0.025	0.931
T_1 artifact file count	Technical complexity	All team-semesters	14	0.271	0.349
T_1 artifact file count	Engagement/participation	All team-semesters	14	−0.079	0.790
T_1 artifact line scope	Project progress	All team-semesters	14	0.096	0.745
T_1 artifact line scope	Scope applicability	All team-semesters	14	0.038	0.897
T_1 artifact line scope	Technical complexity	All team-semesters	14	0.222	0.446
T_1 artifact line scope	Engagement/participation	All team-semesters	14	−0.143	0.626

V. DISCUSSION

The results support a qualified answer to the research questions. Students report frequent GenAI use across development activities, but their perception families do not form a single trajectory. Repository activity and clean source/test change concentrate near final integration, while transcript markers become denser in the observed 2025.2 corpus. RQ3 adds variation in upfront architectural-artifact scope, but no robust monotonic association with late clean-path rework or independently scored outcomes. The central implication is not that larger artifacts guarantee better projects; it is that AI-assisted development makes shared intent and coordination more consequential.

A. From local acceleration to coordination

RQ1 and RQ2 describe two sides of AI-assisted development. Students use GenAI for concrete tasks, yet repository-visible work intensifies near final integration. This weakens a simple productivity story in which faster local code production automatically yields smoother system delivery. A more defensible interpretation is a shifted bottleneck: GenAI may reduce the effort of producing local fragments, but it does not settle architecture, interfaces, task boundaries, dependencies, or acceptance conditions. Those agreements remain coordination objects in multi-person projects.

The robustness analyses also prevent reducing the late peak to one "student syndrome" mechanism. Some team-semesters combine high final concentration with score gains, others show late concentration without evaluator-score gain, and others improve with more distributed activity. The point is not to identify one true mechanism, but to show that repository timing, planning visibility, and evaluator trajectories separate profiles that a simple procrastination narrative would conflate.

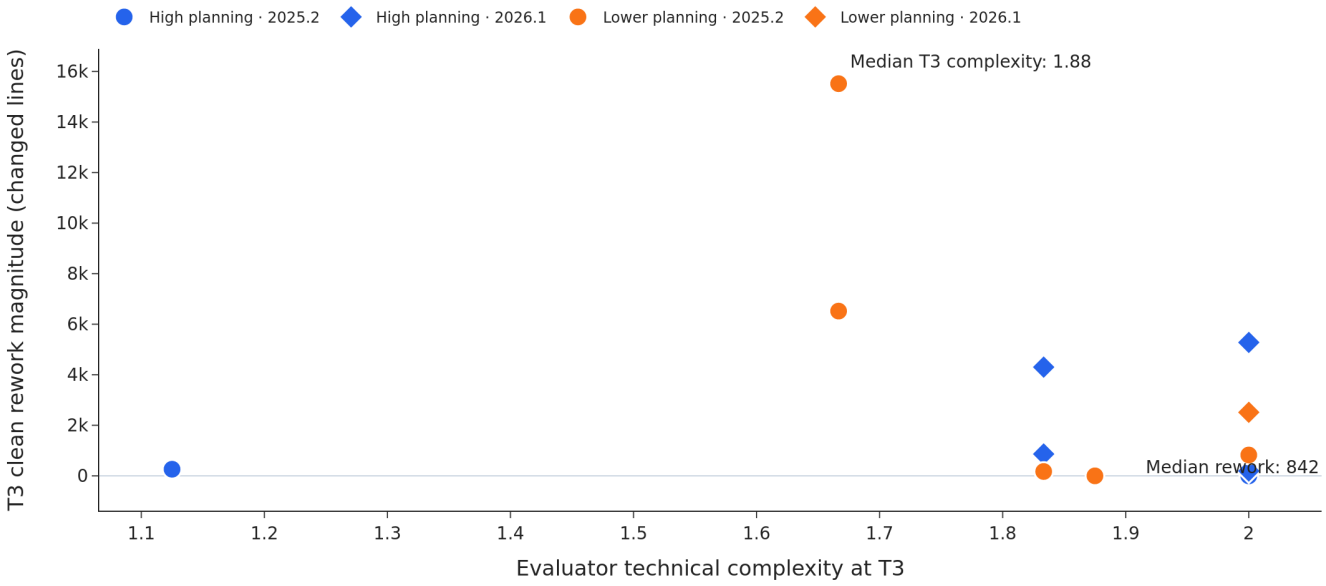


Fig. 7. T3 technical complexity versus clean rework. Technical complexity is an evaluator dimension and clean rework is a clean-path provenance proxy; the figure is descriptive and does not adjust causally for complexity.

B. Planning artifacts, rework, and complexity

RQ3 qualifies strong planning narratives. Architectural Artifact Scope (M6a) captures repository-visible scope, and Planning Content Evidence (M6b) extracts structured evidence from T1 commit subjects; neither measures architectural quality. Repository absence cannot prove that a team did not plan elsewhere, and artifact volume cannot prove that its design was coherent or useful.

Late clean-path rework can reflect project difficulty and structure as well as planning gaps or AI-assisted implementation dynamics. The complexity profiles show why rework should not be attributed to a single source. Likewise, Clean-Path Rework (M8) follows file provenance; it does not determine whether a change was semantically defective or pedagogically productive.

C. Why SDD is a plausible response

The results motivate SDD as a hypothesis for future evaluation, not as a tested treatment. Its specification-to-plan-to-task workflow matches the problem surfaced here: when GenAI makes implementation cheaper while system-level intent remains underspecified, externalizing intent, interfaces, constraints, and acceptance criteria may reduce ambiguity before it spreads across generated code and parallel contributors [1], [2].

This interpretation remains compatible with Agile values. It argues for a clearer object of iteration, not for a frozen plan: specifications can change, but revisions should be explicit, reviewable, and connected to tasks and implementation. For educators, the implication is to assess shared architectural intent, API contracts, cross-component dependencies, design revision rationales, and verifiable acceptance criteria before and during GenAI-assisted implementation. Future classroom studies should pair process metrics with concept assessments and structured reflections to distinguish productive integration struggle from delayed recognition of foundational gaps.

VI. Threats to Validity

The constructs rely on complementary but imperfect proxies. M1 and M2 are survey measures, not behavioral telemetry; M6a and M6b capture repository-visible artifacts and commit subjects, not architectural quality or planning intent; M8 uses file-path provenance rather than semantic defect validation; and M5 uses lexical markers rather than direct friction diagnoses. These boundaries prevent the metrics from being treated as interchangeable measures of quality, effort, or productivity.

The observational design constrains causal and external validity. Late-stage repository concentration remains compatible with ordinary deadline pressure and "student syndrome." We mitigate this threat with internal contrasts by score change, planning scope, non-overlapping phases, and regularity proxies, but without a non-AI control group or timestamped AI-use telemetry we cannot separate GenAI-related debt, conventional procrastination, process choice, and project difficulty. Rolling windows overlap, Git cannot observe off-repository work, and teams freely chose Scrum, Kanban, other Agile variants, or unguided approaches; regularity summaries are descriptive Git proxies, not validated process-quality measures.

Small-sample and coverage limits are substantial. RQ3 covers 14 team-semesters in one educational setting; leave-one-out diagnostics make case influence visible but are not confidence intervals. Analytical grains differ across student responses, team-semesters, rolling windows, and the global transcript corpus, so denominators cannot be pooled implicitly. M5

transcripts are available only for 2025.2; 2026.1 is therefore unavailable, not zero evidence. Five M6b cases lacked T1 commit subjects and were not imputed.

Finally, project complexity remains a plausible confound. The course required an integrated GenAI component but did not mandate RAG or a common architecture, and none of the 14 projects implemented a full RAG pipeline. Teams' self-selected AI integrations differed in technical complexity. Complexity profiles and stratified summaries mitigate this threat descriptively, but cannot isolate causal effects. Likewise, SDD is discussed as a theoretically motivated intervention hypothesis; this study neither implements nor evaluates SDD.

VII. Conclusion

Do we still need to be Agile? Yes, but Generative AI changes what effective agility requires. Iteration, feedback, and adaptation remain valuable; however, faster local implementation does not remove the need for shared architectural intent, explicit interfaces, dependency boundaries, and reviewable acceptance conditions. Across the three research questions, this study shows that self-reported AI use is widespread and task-oriented, repository activity concentrates near final integration, and observable upfront architectural artifacts do not translate into a simple monotonic pattern of rework or project outcomes. The late activity peak is not a single phenomenon: the internal contrasts distinguish cases consistent with compensatory late recovery, cases of late concentration without evaluator-score gain, and cases of more distributed progress.

The study's contribution is to make this tension empirically inspectable through complementary evidence: longitudinal perception and adoption reports, temporal repository measures, transcript-based coordination evidence, structural T1 artifacts, path-based rework measures, and independent evaluator scorecards. The robustness checks do not remove the observational limits of the study, but they make the main validity threats inspectable through non-overlapping phase bins, internal timing/outcome contrasts, operational-regularity proxies, influence diagnostics, complexity profiles, and explicit transcript-coverage boundaries. Together, these results suggest that GenAI-assisted development shifts a larger share of engineering difficulty away from producing local code and toward coordinating system-level intent. The practical implication is not a return to immutable Big Design Up Front–a fixed, comprehensive design completed before implementation–but a stronger planning layer that remains iterative, explicit, and shared by people and AI tools.

Spec-Driven Development is a plausible way to instantiate that planning layer: requirements become specifications, specifications become technical plans and tasks, and implementation is repeatedly checked against those artifacts. This study does not evaluate SDD as an intervention, so its value here is a grounded and testable proposition rather than an established causal conclusion. The case for SDD is that it directly targets the coordination gap made visible by the combined timing, planning, rework, complexity, and influence diagnostics. Future work should compare ordinary practice with an iterative SDD condition while measuring specification quality, consented AI-tool use, integration timing, operational regularity, technical-complexity strata, coordination evidence, rework provenance, and independently scored outcomes.

VIII. Data Availability

The companion static data explorer [27] provides public, double-anonymous review access to the approved research dataset snapshot (`research-data-001`) in downloadable JSON and CSV formats, alongside interactive visualizations and leave-one-out sensitivity maps. In addition to the quantitative tables, the replication package documents (i) the versioned clean-path inclusion/exclusion glob rules (M4/M8), (ii) the complete deterministic lexical dictionary and regular expressions for coordination markers (M5), (iii) the structured extraction prompt and schema for T_1 commit subjects (M6b), and (iv) the full student survey instruments (M1/M2), course checkpoint rubrics, and blind evaluator scorecards (M9). Each dataset preserves its native observation grain and original denominators; no verbatim student meeting transcripts are released to protect participant privacy under institutional ethics guidelines. Upon manuscript acceptance, the complete `research-data-001` snapshot and replication package will be permanently archived with a persistent DOI on Zenodo (to be minted upon camera-ready submission).

References

[1] D. Delimarsky, "Spec-driven development with AI: Get started with a new open source toolkit," https://github.blog/ai-and-ml/generative-ai/spec-driven-development-with-ai-get-started-with-a-new-open-source-toolkit/, sep 2025.

[2] Thoughtworks, "Spec-driven development," November 2025. [Online]. Available: https://www.thoughtworks.com/insights/decoder/s/spec-driven-development

[3] K. Beck, M. Beedle, A. van Bennekum, A. Cockburn, W. Cunningham, M. Fowler, J. Grenning, J. Highsmith, A. Hunt, R. Jeffries, J. Kern, B. Marick, R. C. Martin, S. Mellor, K. Schwaber, J. Sutherland, and D. Thomas, "Manifesto for agile software development," 2001. [Online]. Available: https://agilemanifesto.org/

[4] F. P. Brooks, *The Mythical Man-Month: Essays on Software Engineering*. Addison-Wesley, 1975.

[5] T. DeMarco and T. Lister, *Peopleware: Productive Projects and Teams*. Dorset House, 1987.

[6] A. Carleton, D. Falessi, H. Zhang, and X. Xia, "Generative ai: Redefining the future of software engineering," *IEEE Software*, vol. 41, no. 6, pp. 34–37, 2024.

[7] L. Banh, F. Holldack, and G. Strobel, "Copiloting the future: How generative ai transforms software engineering," *Information and Software Technology*, vol. 183, p. 107751, 2025.

[8] D. Russo *et al.*, "Navigating the complexity of generative ai adoption in software engineering," *ACM Transactions on Software Engineering and Methodology*, vol. 33, 2024.

[9] IEEE Computer Society, "Transforming software engineering processes through generative ai," *IEEE Computer*, July 2025. [Online]. Available: https://www.computer.org/csdl/magazine/co/2025/07/11052608/27RKcQkdNg4

[10] M. Staron and S. Abrahão, "Exploring generative ai in automated software engineering," *IEEE Software*, vol. 42, no. 3, pp. 142–145, 2025.

[11] M. Chen *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021. [Online]. Available: https://arxiv.org/abs/2107.03374

[12] S. Barke, M. B. James, and N. Polikarpova, "Grounded copilot: How programmers interact with code-generating models," *Proceedings of the ACM on Programming Languages*, vol. 7, no. OOPSLA1, pp. 85–111, 2023.

[13] D. Spinellis, B. Baudry, and J. Voas, "Ai: The blind-men elephant in the software developers' room," *Computer*, vol. 59, no. 7, pp. 96–99, 2026.
[14] P. Haindl and G. Weinberger, "Students' experiences of using chatgpt in an undergraduate programming course," *IEEE Access*, vol. 12, pp. 43 519–43 529, 2024.
[15] M. Yabaku and S. Ouhbi, "University students' perception and expectations of generative ai tools for software engineering," in *2024 36th International Conference on Software Engineering Education and Training (CSEE&T)*, 2024, pp. 1–5.
[16] B. Bruegge, S. Krusche, and L. Alperowitz, "Software engineering project courses with industrial clients," *ACM Transactions on Computing Education*, vol. 15, no. 4, pp. 17:1–17:31, 2015.
[17] W. Cunningham, "The wycash portfolio management system," in *OOPSLA '92 Experience Report*. ACM, 1992.
[18] M. Fowler, "Technical debt quadrant," 2009. [Online]. Available: https://martinfowler.com/bliki/TechnicalDebtQuadrant.html
[19] P. Kruchten, R. L. Nord, and I. Ozkaya, "Technical debt: From metaphor to theory and practice," *IEEE Software*, vol. 29, no. 6, pp. 18–21, 2012.
[20] R. Verdecchia, "Architectural technical debt identification: Moving forward," in *2018 IEEE International Conference on Software Architecture Companion (ICSA-C)*, 2018, pp. 43–44.
[21] M. Nayebi, Y. Cai, R. Kazman, G. Ruhe, Q. Feng, C. Carlson, and F. Chew, "A longitudinal study of identifying and paying down architecture debt," in *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 2019, pp. 171–180.
[22] Z. Li, P. Avgeriou, and P. Liang, "A systematic mapping study on technical debt and its management," *Journal of Systems and Software*, vol. 101, pp. 193–220, 2015.
[23] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.
[24] K.-J. Stol and B. Fitzgerald, "The abc of software engineering research," *ACM Transactions on Software Engineering and Methodology*, vol. 27, no. 3, 2018.
[25] P. Runeson and M. Höst, "Guidelines for conducting and reporting case study research in software engineering," *Empirical Software Engineering*, vol. 14, no. 2, pp. 131–164, 2009.
[26] C. Spearman, "The proof and measurement of association between two things," *The American Journal of Psychology*, vol. 15, no. 1, pp. 72–101, 1904.
[27] "Research data explorer," https://moonlit-mochi-1dcef0.netlify.app/#explore, 2026, snapshot: `research-data-001`. Accessed: Oct. 23, 2026.